DA2009 - Group Project - IRCSL Ethical Web Scraping

DA 2009 - Data Collection Methods II
Group 3

Name	Student ID	Phase
Chandrasekaran Sivaneswaran	25ada147	Phase 3 - Brokers and Loss Adjusters
Zainul Abdeen Athif	25ada023	Phase 4 - Document Engine
V. K. Rajapakse	25ada139	Phase 1 - Static Pages
P. P. M. Subasinsani	25ada029	Phase 2 - Insurance Companies

Phase 2 - Insurance Companies Directory (Scrapy)

Done by: P. P. M. Subasinsani (25ada029)
Website: https://ircsl.gov.lk/insurance-companies/

In this phase we collect the list of registered insurance companies (Long Term and General business) with **Scrapy**. We test Scrapy selectors on the page, find the company details hidden in each row, and build a spider with Items, item pipelines and JSON export that also follows a link to a second page.

1. Setup

We use the same User-Agent and 3-second delay as Phase 1, and check robots.txt for the pages we use. Later the spider also checks robots.txt by itself (`ROBOTSTXT_OBEY = True`).

In [1]:
```
import json
import time
import urllib.robotparser

import requests
import pandas as pd
from scrapy.http import TextResponse

USER_AGENT = "DA2009-StudentProject/1.0 (BSc Applied Data Analytics coursework; academic use only)"
HEADERS = {"User-Agent": USER_AGENT}
DELAY_SECONDS = 3

BASE_URL = "https://ircsl.gov.lk"
DIRECTORY_URL = BASE_URL + "/insurance-companies/"

rp = urllib.robotparser.RobotFileParser()
rp.set_url(BASE_URL + "/robots.txt")
rp.read()
print("Directory page allowed by robots.txt:", rp.can_fetch(USER_AGENT, DIRECTORY_URL))
```

2. Exploring the page with Scrapy selectors

We download the page once and load it into Scrapy's `TextResponse`, so we can test Scrapy's CSS and XPath selectors before writing the spider.

- `#company-table tbody tr` selects the rows of the table with `id="company-table"`.
- `td:nth-child(2)` is the second cell (the company name).
- `xpath("string(.)")` gets all the text inside a cell, even if it is inside other tags.

In [2]:
```
response = requests.get(DIRECTORY_URL, headers=HEADERS, timeout=20)
time.sleep(DELAY_SECONDS)
print("Status code:", response.status_code)

scrapy_response = TextResponse(url=DIRECTORY_URL, body=response.text, encoding="utf-8")
rows = scrapy_response.css("#company-table tbody tr")
print("Rows in the table:", len(rows))

for row in rows[:5]:
    print(row.css("td:nth-child(2)").xpath("string(.)").get().strip(), "|",
          " ".join(row.css("td:nth-child(3)").xpath("string(.)").get().split()))
```

```
Status code: 200
Rows in the table: 29
Sri Lanka Insurance Corporation Life Limited ***** | Long Term Insurance
Sri Lanka Insurance Corporation General Limited ***** | General Insurance
Continental Insurance Life Lanka Ltd | Long Term Insurance
Sanasa General Insurance Company Limited | General Insurance
MBSL Insurance Company Limited | Long Term Insurance**** - April 1, 2005 General Insurance - August 2,
2005
```

3. Hidden company details

Our task asks for the head office address and contact people, but the table only shows the name, type and date. In the browser each row has a + button that opens a box with these details.

When we looked at the page source, we found that every row has a small `<script>` tag with a JavaScript list (`new Array(...)`) holding all the details. The page's JavaScript function `format()` uses `values[d][4]`, `values[d][5]` and so on to build the box, which told us what each position means (4 = address, 5 = telephone, 7 = email, 9 = principal officer ...).

To read the list in Python we take the text between `new Array(` and `);` and use `json.loads()`. One item, `img_to_get`, is a JavaScript variable and not text, so we replace it with an empty string first. Because the data is already in the HTML, we did not need a browser to click the buttons.

In [3]:
```
first_row_script = rows[0].css("script::text").get()
list_text = first_row_script.split("new Array(")[1].split(");")[0]
hidden_values = json.loads("[" + list_text.replace("img_to_get", '""') + "]")

labels = ["company_name", "logo", "class", "licence_date", "head_office_address", "telephone",
          "fax", "email", "website", "principal_officer", "specified_officer", "contact_person",
"css_class"]
pd.Series(hidden_values, index=labels)
```

Out[3]:
```
company_name           Sri Lanka Insurance Corporation Life Limited *...
logo
class                                                 Long Term Insurance
licence_date                                             February 1, 2024
head_office_address                      21, Vauxhall Street, Colombo 02
telephone                                0112357709 , Mobile -0771432843
fax                                                           0112357260
email                  ceooffice@srilankainsurance.com, nalin.subasin...
website
principal_officer                                     Mr. Nalin Subasinghe
specified_officer                        Mr. Appukuttige Dayaratne Perera
contact_person
css_class
dtype: str
```

We also checked two other things before writing the spider:

- **Pagination:** the DataTables setting `"iDisplayLength": -1` shows all rows on one page, so there is no next page to follow.
- **Detail pages:** the sitemap lists 29 pages under `/insurance-company/<name>/`. We opened one and it was empty (only the menu and footer), so we did **not** crawl them.

4. Building the Scrapy spider

How Scrapy works:

```
Spider -> Engine -> Scheduler -> Downloader -> Website
Item Pipelines <- Spider <- Engine <- Response
```

The **spider** says where to start and what to extract, the **scheduler** keeps a queue of requests, the **downloader** gets the pages (asynchronously), and the **item pipelines** clean and check every item before it is saved.

We write the spider into a file with `%%writefile` (and `%%writefile -a` to add to it), then run it with `scrapy runspider`.

4.1 Items

A Scrapy Item is like a form with fixed fields, so every record has the same structure. One Item is for companies and one is for the quarterly returns page.

In [4]:
```
%%writefile ircsl_insurance_spider.py
# DA2009 – Group Project – IRCSL Ethical Web Scraping
# DA 2009 – Data Collection Methods II | Group 3
#
# Members:
#   Chandrasekaran Sivaneswaran – 25ada147
#   Zainul Abdeen Athif – 25ada023
#   V. K. Rajapakse – 25ada139
#   P. P. M. Subasinsani – 25ada029
#
# Phase 2 – Insurance Companies spider (Scrapy)
# Done by: P. P. M. Subasinsani (25ada029)
# Run with: scrapy runspider ircsl_insurance_spider.py

import json
import time

import scrapy
from scrapy.exceptions import DropItem
from bs4 import BeautifulSoup

USER_AGENT = "DA2009-StudentProject/1.0 (BSc Applied Data Analytics coursework; academic use only)"
DIRECTORY_URL = "https://ircsl.gov.lk/insurance-companies/"


class InsuranceCompanyItem(scrapy.Item):
```

```
    company_name = scrapy.Field()
    footnote_marker = scrapy.Field()
    class_of_insurance = scrapy.Field()
    licence_effective_date = scrapy.Field()
    registration_number = scrapy.Field()
    head_office_address = scrapy.Field()
    telephone = scrapy.Field()
    fax = scrapy.Field()
    email = scrapy.Field()
    website = scrapy.Field()
    principal_officer = scrapy.Field()
    specified_officer = scrapy.Field()
    contact_person = scrapy.Field()
    source_url = scrapy.Field()
    scraped_at = scrapy.Field()


class QuarterlyReturnItem(scrapy.Item):
    company_name = scrapy.Field()
    within_deadline = scrapy.Field()
    after_deadline = scrapy.Field()
    source_url = scrapy.Field()
    scraped_at = scrapy.Field()
```

```
Writing ircsl_insurance_spider.py
```

4.2 Item pipelines

Every item goes through the pipelines in order (100 first, then 200):

- `CleanTextPipeline` turns HTML codes like `’` back into normal characters, removes extra spaces, and changes empty values and "-" into `None`.
- `ValidateAndDedupePipeline` drops items with no company name and duplicates, using `DropItem`.

In [5]:

```
%%writefile -a ircsl_insurance_spider.py


class CleanTextPipeline:
    def process_item(self, item, spider=None):
        for field, value in item.items():
            if isinstance(value, str):
                if "&" in value or "<" in value:
                    value = BeautifulSoup(value, "html.parser").get_text(" ")
                value = " ".join(value.split())
                if value in ["", "–"]:
                    value = None
                item[field] = value
        return item


class ValidateAndDedupePipeline:
    def __init__(self):
        self.seen = set()

    def process_item(self, item, spider=None):
        if not item.get("company_name"):
            raise DropItem("Missing company name")
        key = (type(item).__name__, item["company_name"], item.get("class_of_insurance"))
        if key in self.seen:
            raise DropItem("Duplicate item")
        self.seen.add(key)
        return item
```

```
Appending to ircsl_insurance_spider.py
```

4.3 The spider, its settings and parse()

- `ROBOTSTXT_OBEY`, `USER_AGENT`, `DOWNLOAD_DELAY = 3` and `CONCURRENT_REQUESTS_PER_DOMAIN = 2` keep the spider polite.
- `FEEDS` saves each type of item to its own JSON file.

- `parse()` reads the visible cells and the hidden list of each row, separates footnote markers at the end of a name (for example "...PLC+" becomes the name and the marker "+"), and uses `yield` to send each item to the pipelines.

In [6]:
```
%%writefile -a ircsl_insurance_spider.py


class InsuranceCompaniesSpider(scrapy.Spider):
    name = "insurance_companies"
    allowed_domains = ["ircsl.gov.lk"]
    start_urls = [DIRECTORY_URL]

    custom_settings = {
        "ROBOTSTXT_OBEY": True,
        "USER_AGENT": USER_AGENT,
        "DOWNLOAD_DELAY": 3,
        "CONCURRENT_REQUESTS_PER_DOMAIN": 2,
        "ITEM_PIPELINES": {CleanTextPipeline: 100, ValidateAndDedupePipeline: 200},
        "FEEDS": {
            "data/processed/insurance_companies.json": {
                "format": "json", "encoding": "utf8", "indent": 2,
                "overwrite": True, "item_classes": [InsuranceCompanyItem],
            },
            "data/processed/quarterly_returns.json": {
                "format": "json", "encoding": "utf8", "indent": 2,
                "overwrite": True, "item_classes": [QuarterlyReturnItem],
            },
        },
        "LOG_LEVEL": "INFO",
    }

    def parse(self, response):
        scraped_at = time.strftime("%Y-%m-%d %H:%M:%S")

        for row in response.css("#company-table tbody tr"):
            name = row.css("td:nth-child(2)").xpath("string(.)").get().strip()

            # hidden details inside the <script> tag of the row
            hidden = [""] * 13
            script = row.css("script::text").get() or ""
            if "new Array(" in script:
                list_text = script.split("new Array(")[1].split(");")[0]
                hidden = json.loads("[" + list_text.replace("img_to_get", '""') + "]")

            # footnote markers such as ***** or + at the end of a name
            clean_name = name.rstrip("*+ ")
            marker = name[len(clean_name):].strip()

            yield InsuranceCompanyItem(
                company_name=clean_name,
                footnote_marker=marker if marker else None,
                class_of_insurance=row.css("td:nth-child(3)").xpath("string(.)").get(),
                licence_effective_date=row.css("td:nth-child(4)").xpath("string(.)").get(),
                registration_number=None,
                head_office_address=hidden[4],
                telephone=hidden[5],
                fax=hidden[6],
                email=hidden[7],
                website=hidden[8],
                principal_officer=hidden[9],
                specified_officer=hidden[10],
                contact_person=hidden[11],
                source_url=response.url,
                scraped_at=scraped_at,
            )
```

```
Appending to ircsl_insurance_spider.py
```

4.4 Following a link to a second page

The website menu links to the "Submission of Quarterly Returns" page, which lists the insurers again with the date of their last return. We find the link with XPath, make it a full URL with `response.urljoin()`, and add it to Scrapy's queue with

`scrapy.Request(..., callback=...)`. The table on that page has no id, so we find it by its header text.

In [7]:
```
%%writefile -a ircsl_insurance_spider.py

        # follow the link to the Quarterly Returns page
        href = response.xpath(
            '//a[contains(@href, "/insurance-companies/submission-of-quarterly-returns")]/@href'
        ).get()
        if href:
            next_url = response.urljoin(href)
            self.logger.info("Following link: " + next_url)
            yield scrapy.Request(next_url, callback=self.parse_quarterly_returns)

    def parse_quarterly_returns(self, response):
        scraped_at = time.strftime("%Y-%m-%d %H:%M:%S")

        for table in response.css("table"):
            header = table.css("tr")[0].xpath("string(.)").get()
            if "Name of Insurance Company" not in header:
                continue
            for row in table.css("tr")[1:]:
                cells = [c.xpath("string(.)").get().strip() for c in row.css("td")]
                if len(cells) < 4:
                    continue
                yield QuarterlyReturnItem(
                    company_name=cells[1],
                    within_deadline=cells[2],
                    after_deadline=cells[3],
                    source_url=response.url,
                    scraped_at=scraped_at,
                )
```

```
Appending to ircsl_insurance_spider.py
```

4.5 Running the spider

In the log we can see robots.txt being checked first, the "Following link" message, and the final stats: `downloader/request_count` (requests made) and `item_scraped_count` (items saved).

In [8]:
```
!mkdir -p data/processed
!scrapy runspider ircsl_insurance_spider.py
```

```
2026-10-09 11:51:05 [scrapy.utils.log] INFO: Scrapy 2.19.0 started (bot: scrapybot)
2026-10-09 11:51:05 [scrapy.utils.log] INFO: Versions:
{'lxml': '6.1.3',
 'libxml2': '2.14.6',
 'cssselect': '1.5.0',
 'parsel': '1.12.1',
 'w3lib': '2.5.0',
 'Twisted': '26.4.0',
 'Python': '3.11.7 (main, Dec 15 2023, 12:09:56) [Clang 14.0.6 ]',
 'pyOpenSSL': '26.4.0 (OpenSSL 4.0.3 29 Sep 2026)',
 'cryptography': '50.0.2',
 'Platform': 'macOS-26.6.1-arm64-arm-64bit'}
2026-10-09 11:51:05 [scrapy.crawler] DEBUG: Using AsyncCrawlerProcess
2026-10-09 11:51:05 [asyncio] DEBUG: Using selector: KqueueSelector
2026-10-09 11:51:05 [scrapy.addons] INFO: Enabled addons:
[]
2026-10-09 11:51:05 [scrapy.extensions.telnet] INFO: Telnet Password: 6e603bf28fc6f10a
2026-10-09 11:51:06 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.logcount.LogCount',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.memusage.MemoryUsage',
 'scrapy.extensions.feedexport.FeedExporter',
 'scrapy.extensions.logstats.LogStats',
 'scrapy.extensions.remote_control.RemoteControl']
2026-10-09 11:51:06 [scrapy.crawler] INFO: Overridden settings:
{'CONCURRENT_REQUESTS_PER_DOMAIN': 2,
 'DOWNLOAD_DELAY': 3,
 'LOG_LEVEL': 'INFO',
 'ROBOTSTXT_OBEY': True,
 'SPIDER_LOADER_CLASS': <class 'scrapy.spiderloader.DummySpiderLoader'>,
 'USER_AGENT': 'DA2009-StudentProject/1.0 (BSc Applied Data Analytics '
               'coursework; academic use only)'}
2026-10-09 11:51:06 [scrapy.middleware] INFO: Enabled downloader middlewares:
['scrapy.downloadermiddlewares.offsite.OffsiteMiddleware',
 'scrapy.downloadermiddlewares.robotstxt.RobotsTxtMiddleware',
 'scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
 'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
 'scrapy.downloadermiddlewares.redirect.RedirectMiddleware',
 'scrapy.downloadermiddlewares.cookies.CookiesMiddleware',
 'scrapy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
 'scrapy.downloadermiddlewares.stats.DownloaderStats']
2026-10-09 11:51:06 [scrapy.middleware] INFO: Enabled spider middlewares:
['scrapy.spidermiddlewares.start.StartSpiderMiddleware',
 'scrapy.spidermiddlewares.httperror.HttpErrorMiddleware',
 'scrapy.spidermiddlewares.referer.RefererMiddleware',
 'scrapy.spidermiddlewares.urllength.UrlLengthMiddleware',
 'scrapy.spidermiddlewares.depth.DepthMiddleware',
 'scrapy.spidermiddlewares.metacopy.MetaCopyDetectionMiddleware']
2026-10-09 11:51:06 [scrapy.middleware] INFO: Enabled item pipelines:
[<class 'ircsl_insurance_spider.CleanTextPipeline'>,
 <class 'ircsl_insurance_spider.ValidateAndDedupePipeline'>]
2026-10-09 11:51:06 [scrapy.core.engine] INFO: Spider opened
2026-10-09 11:51:06 [scrapy.extensions.logstats] INFO: Crawled 0 pages (at 0 pages/min), scraped 0 items
(at 0 items/min)
2026-10-09 11:51:06 [scrapy.extensions.telnet] INFO: Telnet console listening on 127.0.0.1:6023
2026-10-09 11:51:06 [scrapy.extensions.remote_control] INFO: Remote control HTTP server listening on port
60731 (job 12021-efa54e4333d74dfaabdb2a4c5f5580cb)
2026-10-09 11:51:10 [insurance_companies] INFO: Following link: https://ircsl.gov.lk/quick-
links/regulatory-reporting/compliance-with-regulatory-reporting/insurance-companies/submission-of-
quarterly-returns/
2026-10-09 11:51:15 [scrapy.core.engine] INFO: Closing spider (finished)
2026-10-09 11:51:15 [scrapy.extensions.feedexport] INFO: Stored json feed (29 items) in:
data/processed/insurance_companies.json
2026-10-09 11:51:15 [scrapy.extensions.feedexport] INFO: Stored json feed (28 items) in:
data/processed/quarterly_returns.json
2026-10-09 11:51:15 [scrapy.statscollectors] INFO: Dumping Scrapy stats:
{'downloader/request_bytes': 1020,
 'downloader/request_count': 3,
 'downloader/request_method_count/GET': 3,
 'downloader/response_bytes': 39380,
 'downloader/response_count': 3,
 'downloader/response_status_count/200': 3,
 'elapsed_time_seconds': 9.444421374995727,
 'feedexport/success_count/FileFeedStorage': 2,
 'finish_reason': 'finished',
 'finish_time': datetime.datetime(2026, 10, 9, 6, 21, 15, 591313, tzinfo=datetime.timezone.utc),
```

```
'httpcompression/response_bytes': 198867,
'httpcompression/response_count': 3,
'item_scraped_count': 57,
'items_per_minute': 380.0,
'log_count/INFO': 5,
'memusage/max': 87228416,
'memusage/startup': 87228416,
'request_depth_max': 1,
'response_received_count': 3,
'responses_per_minute': 20.0,
'robotstxt/request_count': 1,
'robotstxt/response_count': 1,
'robotstxt/response_status_count/200': 1,
'scheduler/dequeued': 2,
'scheduler/dequeued/memory': 2,
'scheduler/enqueued': 2,
'scheduler/enqueued/memory': 2,
'start_time': datetime.datetime(2026, 10, 9, 6, 21, 6, 147173, tzinfo=datetime.timezone.utc)}
2026-10-09 11:51:15 [scrapy.core.engine] INFO: Spider closed (finished)
```

5. Checking and cleaning the data

Registration number is empty because IRCSL does not publish it, and contact person is almost always empty because most companies only list a principal and a specified officer.

In [9]:
```
companies = pd.read_json("data/processed/insurance_companies.json")
quarterly = pd.read_json("data/processed/quarterly_returns.json")
print("Companies:", len(companies), "| quarterly return rows:", len(quarterly))

completeness = companies.notna().sum().to_frame("values_found")
completeness["percent"] = (completeness["values_found"] / len(companies) * 100).round(0)
completeness.T
```

```
Companies: 29 | quarterly return rows: 28
```

Out[9]:

	company_name	footnote_marker	class_of_insurance	licence_effective_date	registration_number	head_office_address	telephone	fax	email	website	principal_officer	specified_officer	contact_person	source_url	scraped_at
values_found	29.0	6.0	29.0	28.0	0.0	29.0	29.0	27.0	27.0	25.0	28.0	28.0	2.0	29.0	29.0
percent	100.0	21.0	100.0	97.0	0.0	100.0	100.0	93.0	93.0	86.0	97.0	97.0	7.0	100.0	100.0

One row was broken: MBSL Insurance has two licences written in one cell and an empty date column (`Long Term Insurance**** – April 1, 2005 General Insurance – August 2, 2005`). We split this into the two classes and two dates, and then changed the licence date into a real date with `pd.to_datetime()` .

In [10]:
```
broken = companies["licence_effective_date"].isna()
for i in companies[broken].index:
    parts = companies.at[i, "class_of_insurance"].replace("*", "").split(" – ")
    companies.at[i, "class_of_insurance"] = "Long Term Insurance; General Insurance"
    companies.at[i, "licence_effective_date"] = parts[1].replace("General Insurance", "").strip() + ";
" + parts[2].strip()

first_dates = companies["licence_effective_date"].str.split(";").str[0].str.strip()
companies["first_licence_date"] = pd.to_datetime(first_dates, format="%B %d, %Y")
companies.loc[broken, ["company_name", "class_of_insurance", "licence_effective_date",
"first_licence_date"]]
```

Out[10]:

	company_name	class_of_insurance	licence_effective_date	first_licence_date
4	MBSL Insurance Company Limited	Long Term Insurance; General Insurance	April 1, 2005; August 2, 2005	2005-04-01

6. Joining with the quarterly returns and saving

The two pages write some names differently ("AIA Insurance Lanka Ltd" and "AIA Insurance Lanka PLC"), so we made a `name_key` without common words (PLC, Ltd,

Limited, Company) and punctuation, and joined the two tables on it.

In [11]:
```
def make_name_key(name):
    words = str(name).lower().replace("’", "").replace("'", "").replace(".", " ").split()
    keep = [w for w in words if w not in ["plc", "ltd", "limited", "company", "co", "the"]]
    return "".join(ch for ch in "".join(keep) if ch.isalpha())


companies["name_key"] = companies["company_name"].apply(make_name_key)
quarterly["name_key"] = quarterly["company_name"].apply(make_name_key)
final = companies.merge(quarterly[["name_key", "within_deadline", "after_deadline"]], on="name_key",
how="left")

matched = final["within_deadline"].notna() | final["after_deadline"].notna()
print("Matched with the quarterly returns page:", matched.sum(), "of", len(final))
print("Not found:", final.loc[~matched, "company_name"].tolist())

final = final.drop(columns="name_key")
final.to_csv("data/processed/insurance_companies_clean.csv", index=False)
final.to_json("data/processed/insurance_companies_clean.json", orient="records", indent=2,
date_format="iso")
final[["company_name", "class_of_insurance", "first_licence_date", "head_office_address",
"principal_officer", "within_deadline"]].head()
```

```
Matched with the quarterly returns page: 28 of 29
Not found: ['Arpico Insurance PLC']
```

Out[11]:

	company_name	class_of_insurance	first_licence_date	head_office_address	principal_officer	within_deadline
0	Sri Lanka Insurance Corporation Life Limited	Long Term Insurance	2024-02-01	21, Vauxhall Street, Colombo 02	Mr. Nalin Subasinghe	30/7/2026
1	Sri Lanka Insurance Corporation General Limited	General Insurance	2024-02-01	21, Vauxhall Street, Colombo 02	Dr. Sameera Dharmasena	30/7/2026
2	Continental Insurance Life Lanka Ltd	Long Term Insurance	2022-07-29	No. 79 C W W Kannangara Mawatha, Colombo 07	Mr. H. V. Saliya Dias	30/7/2026
3	Sanasa General Insurance Company Limited	General Insurance	2019-07-01	No. 172, Elvitigala Mawatha, Colombo 08.	Mr. Lihiniya Kumarage Wasantha Premalal Perera	30/7/2026
4	MBSL Insurance Company Limited	Long Term Insurance; General Insurance	2005-04-01	No. 519, T.B. Jayah Mawatha, Colombo 10	Ms. J. Surekha Alles	NaN

7. Ethical considerations and limitations

- robots.txt was checked with `urllib.robotparser` and also by Scrapy (`ROBOTSTXT_OBEY = True`).
- We used a 3-second delay, at most 2 requests at the same time, and our student User-Agent.
- The spider made only 3 requests. We checked one detail page and decided not to crawl the other empty pages.
- We read the hidden data from the HTML instead of clicking every button with a browser, which puts less load on the server.
- The officer names and contact details are official information published by IRCSL, used only for this assignment. Every row keeps its `source_url` and `scraped_at`.
- **Limitations:** IRCSL does not publish registration numbers, so that column is empty. Arpico Insurance PLC is in the directory but not on the Quarterly Returns page.